

Kubernetes

1. Kubernetes Nedir?

Kubernetes, hem bildirimsel (declarative) yapılandırmayı hem de otomasyonu kolaylaştıran, konteynerleştirilmiş iş yüklerini ve hizmetlerini yönetmek için taşınabilir, genişletilebilir, açık kaynaklı bir platformdur. Geniş ve hızla büyüyen bir ekosisteme sahiptir. Kubernetes hizmetleri, desteği ve araçları yaygın olarak mevcuttur.

Kubernetes adı, Yunanca da dümenci veya pilot anlamına gelir. K8S kısaltması, “K” ve “s” harflerinin arasındaki sekiz harfin sayılmasıyla elde edilir.

Kubernetes, Google tarafından 6 Haziran 2014’te duyuruldu. Proje, Google çalışanları Joe Beda, Brendan Burns ve Craig McLuckie tarafından tasarlandı ve oluşturuldu. 1.0 versiyonunu 21 Temmuz 2015’te yayınladı. Google, Cloud Native Computing Foundation (CNCF) oluşturmak için Linux Foundation ile çalıştı ve Kubernetes’i başlangıç teknolojisi olarak sundu.

2. Neden Kubernetes’e İhtiyacınız Var?

Konteynerler, uygulamalarınızı paketlemek ve çalıştırmak için iyi bir yoldur. Üretim ortamında, uygulamaları çalıştıran konteynerleri yönetmeniz ve kesinti olmamasını sağlamanız gerekir. Örneğin, bir konteyner çökerse, başka bir konteynerin başlatılması gerekir. Bu davranışın bir sistem tarafından yönetilmesi daha kolay olmaz mıydı?

İşte Kubernetes burada devreye giriyor! Kubernetes, dağıtılmış sistemleri dayanıklı bir şekilde çalıştırmak için bir çerçeve sunar. Uygulamanız için ölçeklendirme ve arıza durumunda yedekleme işlemlerini üstlenir, dağıtım modelleri sağlar ve daha fazlasını sunar.

Kubernetes size şunları sağlar:

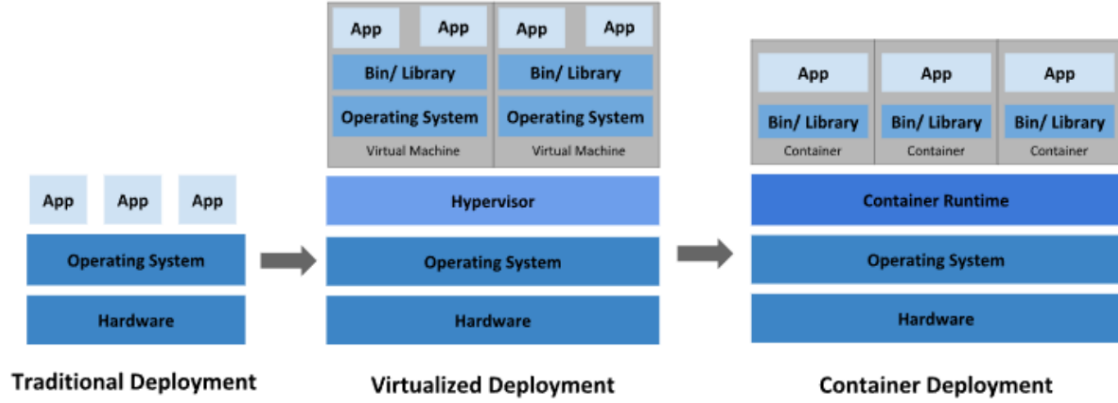
- **Service Discovery and Load Balancing (Servis Keşfi ve Yük Dengeleme):**
Kubernetes, bir konteyneri DNS adı veya IP adresi kullanarak kullanıma sunabilir.

Bir konteynere gelen trafik yüksekse, Kubernetes ağ trafiğini yük dengeleyerek ve dağıtarak dağıtımın istikrarlı olmasını sağlar.

- **Storage Orchestration (Depolama Orkestrasyonu):** Kubernetes, yerel depolama, genel bulut sağlayıcıları ve daha fazlası gibi seçtiğiniz bir depolama sistemini otomatik olarak bağlamanıza olanak tanır.
- **Automated Rollouts and Rollbacks (Otomatik Dağıtım ve Geri Alma):** Kubernetes kullanarak dağıtılan konteynerleriniz için istenen durumu tanımlayabilir ve gerçek durumu kontrollü bir hızda istenen duruma değiştirebilirsiniz. Örneğin, Kubernetes dağıtımınız içinde yeni konteynerler oluşturmak, mevcut konteynerleri kaldırmak ve tüm kaynaklarını yeni konteynere aktarmak için otomatikleştirebilirsiniz.
- **Automatic bin packing:** Kubernetes'e konteynerleştirilmiş görevleri çalıştırmak için kullanabileceği bir düğüm kümesi (node cluster, worker cluster, worker node) sağlarsınız. Kubernetes'e her konteynerin ne kadar CPU ve belleğe ihtiyacı olduğunu söylersiniz. Kubernetes, kaynaklarınızdan en iyi şekilde yararlanmak için konteynerleri düğümlerinize yerleştirebilir.
- **Self-Healing (Kendi kendini onarma):** Kubernetes, başarısız olan konteynerleri yeniden başlatır, konteynerleri değiştirir, kullanıcı tanımlı sağlık kontrolünüze yanıt vermeyen konteynerleri öldürür ve hizmet vermeye hazır olana kadar istemcilere duyurmaz.
- **Secret and Configuration Management (Gizlilik ve Yapılandırma Yönetimi):** Kubernetes parolalar, OAuth token'ları ve SSH anahtarları gibi hassas bilgileri saklamanıza ve yönetmenize olanak tanır. Konteyner görüntülerinizi yeniden oluşturmadan ve yapılandırmalarınızda gizlilikleri açığa çıkarmadan uygulama yapılandırmasını dağıtabilir ve güncelleyebilirsiniz.
- **Batch Execution (Toplu İşleme):** Servislere ek olarak, Kubernetes, istenirse başarısız olan konteynerleri değiştirerek toplu işlem ve CI iş yüklerinizi yönetebilir.
- **Horizontal Scaling or Vertical Scaling(Yatay Ölçeklendirme veya Dikey Ölçeklendirme):** Uygulamanızı basit bir komutla, kullanıcı arayüzüyle veya CPU kullanımına bağlı olarak otomatik olarak yukarı ve aşağı ölçeklendirin
- **IPv4/IPv6:** Pod'lara ve servislere IPv4 ve IPv6 adreslerinin tahsis edilmesi.

- **Designed for Extensibility (Geniřletilebilirlik için Tasarlanmıřtır):** Yukarı akıř kaynak kodunu deęiřtirmeden Kubernetes kümenize özellikler ekleyin.

3. Kubernetes'in Tarihsel Geliřim Süreci



řekil 2.1.

3.1. Geleneksel Daęıtım Dönemi (Traditional Deployment)

Kuruluřlar, bařlangıçta uygulamalarını fiziksel sunucular çalıřtırıyordu. Fiziksel bir sunucudaki uygulamalar için kaynak sınırlarını tanımlamanın bir yolu yoktur ve bu da kaynak tahsisi sorunlarına yol açıyordu. Örneęin fiziksel bir sunucuda birden fazla uygulama çalıřıyorsa, bir uygulamanın kaynakların çoęunun tükettięi ve bunun sonucunda dięer uygulamaların düşük performans gösterdięi durumlar olabilir. Bu soruna çözüm olarak geleneksel daęıtım döneminde, her uygulamayı farklı bir fiziksel sunucuda çalıřtırmaktı. Ancak her bir fiziksel sunucuya kurulan uygulamalar sunucu ięerisindeki kaynakların tamamını kullanmadıkları için sunucularda atıl kapasite durumlarının ortaya çıkması ve kuruluřların her bir uygulama için kurmuř oldukları fiziksel sunucunun yönetimi maliyetli bir durum olduęu için bu yöntem ölçeklenemedi.

3.2. Sanallařtırılmıř Daęıtım Dönemi (Virtualized Deployment)

Çözüm olarak sanallařtırma tanıtıldı. Sanallařtırma, tek bir fiziksel sunucunun CPU'sunda birden fazla sanal makine çalıřtırmamıza olanak tanır. Sanallařtırma, uygulamaların sanal ortamlarda izole edilmesi olanak tanır ve bir uygulamanın bilgilerine bařka bir uygulama serbestçe eriřemedięinden belirli bir düzeyde güvenlik saęlar.

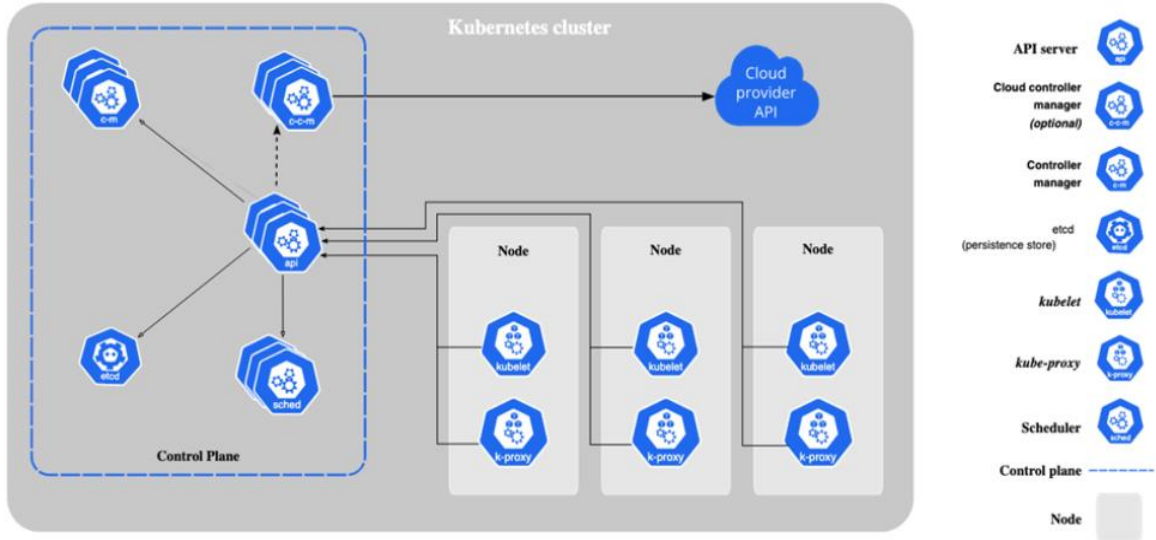
Sanallaştırma, fiziksel bir sunucudaki kaynakların daha iyi kullanılmasını sağladığı için donanım maliyetlerinin düşürülmesinde çok önemli bir rol oynar.

3.3. Konteyner Dağıtım Dönemi (Container Deployment)

Konteynerler sanal makinelere benzer, ancak işletim sistemini uygulamalar arasında paylaşmak için esnek yalıtım özelliklerine sahiptirler. Bu nedenle konteynerler hafif olarak kabul edilir. Konteynerler ekstra faydalar sağladıkları için daha popüler hale geldiler:

- **Agile Application Creation and Deployment (Çevik Uygulama Oluşturma ve Dağıtım):** Sanal makine imajı kullanımına kıyasla konteyner imajı oluşturmanın kolaylığı ve verimliliği artar.
- **Continuous Development, Integration and Deployment (Sürekli Geliştirme, Entegrasyon ve Dağıtım):** Güvenilir ve sık konteyner imajı oluşturma ve dağıtımı sağlar ve hızlı ve verimli geri alma işlemleri sunar.
- **Dev and Ops Separation of Concerns (Geliştirme ve Operasyon Sorumluluklarının Ayrılması):** Uygulama konteyner imajlarını dağıtım zamanında değil, derleme zamanında oluşturarak uygulamaları atyapıdan ayırır.
- **Observability (Gözlemlenebilirlik):** Yalnızca işletim sistemi düzeyindeki bilgileri ve metrikleri değil, aynı zamanda uygulama sağlığını ve diğer sinyalleri de ortaya çıkarır.
- **Environmental Consistency Across Development, Testing and Production (Geliştirme, Test ve Üretimde Ortam Tutarlılığı):** Dizüstü bilgisayarlarda olduğu gibi bulutta da aynı şekilde çalışır.
- **Cloud and OS Distribution Portability (Bulut ve İşletim Sistemi Dağıtım Taşınabilirliği):** Ubuntu, RHEL, CoreOS, şirket içi, büyük genel bulutlarda ve başka herhangi bir yerde çalışır.
- **Application-centric Management (Uygulama Merkezli Yönetim):** Bir işletim sistemini sanal donanımda çalıştırmaktan, bir uygulamayı mantıksal kaynaklar kullanarak bir işletim sisteminde çalıştırmaya kadar soyutlama seviyesini yükseltir.
- **Loosely Coupled, distributed, elastic, liberated micro-services (Gevşek bağlantılı, dağıtılmış, esnek, özgürleştirilmiş mikro servisler):** Uygulamalar daha küçük, bağımsız parçalara ayrılır ve dinamik olarak dağıtılabilir ve yönetilebilir.
- **Resource Isolation (Kaynak İzolasyonu):** Öngörülebilir uygulama performansı
- **Resource Utilization (Kaynak Kullanımı):** Yüksek verimlilik ve yoğunluk.

4. Kubernetes Komponentleri



Şekil 4.1.

4.1. Temel Bileşenler

Bir Kubernetes kümesi (cluster), Bir Control Plane (Master-Node) ve bir veya daha fazla worker node'dan oluşur.

4.1.1. Control Plane (Kontrol Düzlemi- Master Node) Component

Küme hakkında genel kararların alındığı komponentleri barındırır.

4.1.1.1. kube-apiserver

Kubernetes'in merkezi API sunucusudur. Kullanıcılar ve diğer tüm bileşenler sadece kube-apiserver üzerinden iletişim kurarlar.

4.1.1.2. etcd

Tüm API sunucu verileri için tutarlı ve yüksek kullanılabilirliğe sahip anahtar-değer deposu.

4.1.1.3. scheduler

Kubernetes kümesinde oluşturulan ve henüz node ataması yapılmayan pod'ların kısıtlara göre hangi node üzerinde çalışacağını belirleyen kontrol düzlemi bileşenidir.

4.1.1.4. kube-controller-manager

Kubernetes kümesinin gerçek durumu ile istenilen durumların sürekli olarak uyumlu tutmak için çalışan kubernetes komponentidir.

4.1.1.5. cloud-controller-manager

Altta yatan bulut sağlayıcı/sağlayıcıları ile entegre olur.

4.1.2. Node (İşçi Düğüm- Worker Node) Component

Pod'ların çalıştığı makinedir.

4.1.2.1. kubelet

Pod içerisinde tanımlanan konteynerlerin başlatılmasını, çalışır durumda olmasını ve sağlık durumlarını kontrol eder.

4.1.2.2. kube-proxy

Servislerin uygulanması için node'larda ağ kurallarını korur.

4.1.3. Addons (Eklentiler)

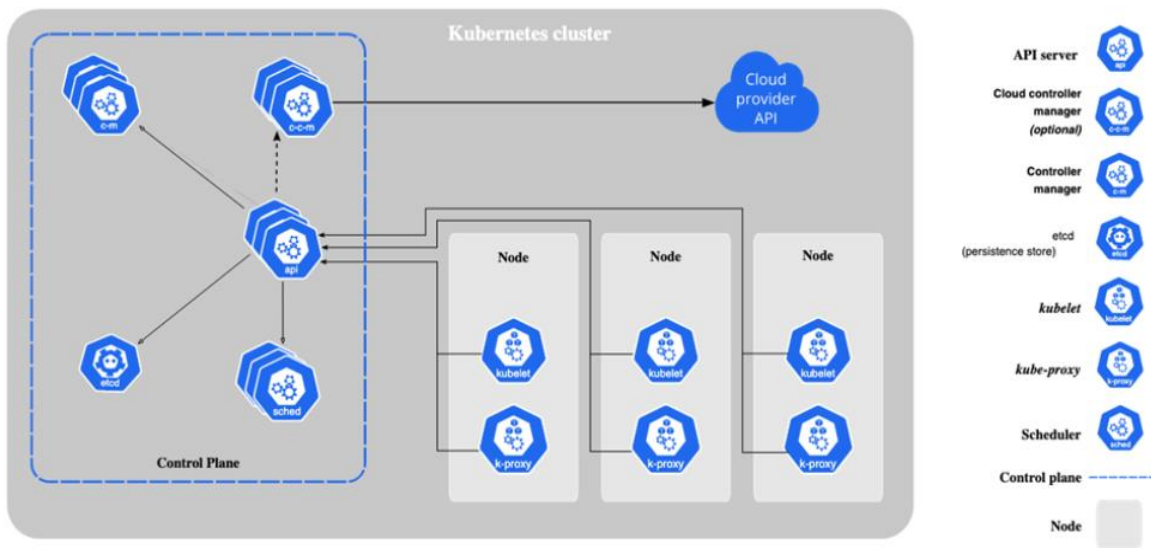
Eklentiler Kubernetes'in işlevselliğini genişletir. Birkaç önemli örnek şunlardır:

- **DNS:** Küme genelinde DNS çözümlemesi için.
- **WebUI:** Web arayüzü üzerinden küme yönetimi için.
- **Container Resource Monitoring:** Konteyner ölçümlerini toplamak ve saklamak için.

5. Kubernetes Küme Mimarisi

Bir Kubernetes kümesi, master node ve konteynerleştirilmiş uygulamaları çalıştıran worker node(lardan) adı verilen makinelerden oluşur. Her kümenin pod'ları çalıştırmak için en az bir worker node'a ihtiyacı vardır.

Worker node, uygulama iş yükünün bileşenleri olan Pod'ları barındırır. Master node, kümedeki worker node ve pod'ları yönetir. Üretim ortamlarında, master node genellikle birden fazla bilgisayarda çalışır ve bir küme genellikle birden fazla düğüm çalıştırarak hata toleransı ve yüksek kullanılabilirlik sağlar.



Şekil 5.1.

5.1. Control Plane (Kontrol Düzlemi- Master Node) Component

Kontrol düzlemi bileşenleri, küme hakkında küresel kararlar alır ve küme olaylarını algılar ve bunlara yanıt verir.

5.1.1. kube-apiserver

Kubernetes API sunucusunun ana uygulaması kube-apiserver'dır. Kube-apiserver yatay ölçeklenecek şekilde tasarlanmıştır; yani daha fazla örnek dağıtarak ölçeklenir. Birkaç kube-apiserver örneği çalıştırabilir ve bu örnekler arasında trafiği dengeleyebilirsiniz.

5.1.2. etcd

Kubernetes'in tüm küme verileri için depolama alanı olarak kullanılan, tutarı ve yüksek kullanılabilirliğe sahip anahtar-değer deposu.

5.1.3. kube-scheduler

Yeni oluşturulan ve henüz bir düğüme atanmamış Pod'ları izleyen ve bunların çalışacağı düğümü seçen kontrol düzlemi bileşenidir.

Planlama kararlarında dikkate alınan faktörler şunlardır: bireysel ve toplu kaynak gereksinimleri, donanım/yazılım/politika kısıtlamaları, yakınlık ve karşıt yakınlık özellikleri, veri yerelliği, iş yükler arası etkileşim ve son teslim tarihleri.

5.1.4. kube-controller-manager

Kontrol düzlemi bileşeni olup, kontrol süreçlerini çalıştırır.

Mantıksal olarak, her kontrolcü ayrı bir süreçtir, ancak karmaşıklığı azaltmak için hepsi tek bir ikili dosyaya derlenir ve tek bir süreçte çalıştırılır. Birçok farklı kontrolcü türü vardır. Bunlardan bazıları şunlardır:

- **Node Controller:** Düğümlerin devre dışı kaldığını fark etmekten ve yanıt vermekten sorumludur.
- **Job Controller:** Tek seferlik görevleri temsil eden "Job" nesnelerini izler ve çalıştırılmasını sağlar.
- **EndpointSlice Controller:** Servis ve podları eşler.
- **Replication Controller:** Pod sayısının Deployment/Statefulset konfigürasyon dosyasında belirtildiği gibi kopyalarının oluşturulup oluşturulmadığını kontrol eder
- **Service Account Controller:** Yeni namespace'ler için ServiceAccount nesnesi oluşturur.

5.1.5 cloud-controller-manager

Buluta özgü kontrol mantığını yerleştiren bir Kubernetes kontrol düzlemi bileşeni. cloud-controller-manager, kümenizi bulut sağlayıcınızın API'sine bağlamanıza ve bu bulut platformuyla etkileşim kuran bileşenleri, yalnızca kümenizle etkileşim kuran bileşenlerden ayırmanıza olanak tanır. cloud-controller-manager, yalnızca bulut sağlayıcınıza özgü denetleyicileri çalıştırır. Kubernetes'i kendi tesislerinizde veya kendi bilgisayarınızdaki bir öğrenme ortamında çalıştırıyorsanız, kümenin bir bulut denetleyici yöneticisi yoktur.

kube-controller-manager da olduğu gibi, cloud controller manager da mantıksal olarak bağımsız birkaç kontrol döngüsünü tek bir işlem olarak çalıştırdığınız tek bir ikili dosyada birleştirir. Performansı artırmak veya arızalara karşı toleransı artırmak için yatay olarak ölçeklendirebilirsiniz (birden fazla kopya çalıştırabilirsiniz).

Aşağıdaki denetleyicilerin bulut sağlayıcı bağımlılıkları olabilir:

- **Node Controller:** Bir node'un yanıt vermeyi bırakmasının ardından bulutta silinip silinmediğini belirlemek için bulut sağlayıcısını kontrol etmek için kullanılır.
- **Route Controller:** Temel bulut altyapısında rotaları ayarlamak için kullanılır.
- **Service Controller:** Bulut sağlayıcı yük dengeleyicilerini oluşturmak, güncellemek ve silmek için kullanılır.

5.2. Node (İşçi Düğüm- Worker Node)

Düğüm bileşenleri her düğümde çalışır, çalışan pod'ları sürdürür ve Kubernetes çalışma ortamını sağlar.

5.1.1. kubelet

Kümedeki her düğümde çalışır. Konteynerlerin bir pod içinde çalıştığından emin olur. Kubelet, Kubernetes tarafından oluşturulmayan kapsayıcıları yönetmez.

5.1.2. kube-proxy

kube-proxy, düğümlerde ağ kurallarını korur. Bu ağ kuralları, kümenizin içindeki veya dışındaki ağ oturumlarından Pod'larınıza ağ iletişimi sağlar. Servisler için paket iletimini kendi başına uygulayan ve kube-proxy'ye eşdeğer davranış sağlayan bir ağ eklentisi kullanıyorsanız, kümenizdeki düğümlerde kube-proxy çalıştırmanıza gerek yoktur.

5.1.3. container runtime

Kubernetes'in konteynerleri etkili bir şekilde çalıştırmasını sağlayan temel bir bileşendir. Kubernetes ortamında konteynerlerin yürütülmesini ve yaşam döngüsünü yönetmekten sorumludur.

Kubernetes, containerd, CRI-O ve Kubernetes CRI'nin (Konteyner Çalışma Zamanı Arayüzü) diğer tüm uygulamaları gibi konteyner çalışma zamanlarını destekler.

6. Containers (Konteynerler)

Uygulamanın çalışma zamanı bağımlılıklarıyla birlikte paketlenmesi için kullanılan teknoloji.

Bu sayfada konteynerler ve konteyner imajları ile bunların operasyon ve çözüm geliştirmedeki kullanımları ele alınacaktır.

“Konteyner” kelimesi çok anlam yüklü bir terimdir. Bu kelimeyi kullandığınızda, hedef kitlenizin aynı tanımı kullanıp kullanmadığını kontrol edin.

Çalıştırdığınız her konteyner tekrarlanabilir, bağımlılıkların dahil edilmesinden kaynaklanan standardizasyon, nerede çalıştırırsanız çalıştırın aynı davranışı elde etmenizi sağlar.

Konteynerler, uygulamaları altta yatan ana bilgisayar altyapısından ayırır. Bu, farklı bulut veya işletim sistemi ortamlarında dağıtımı kolaylaştırır.

Bir Kubernetes kümesindeki her düğüm, o düğüme atanan Pod'ları oluşturan konteynerleri çalıştırır. Bir Pod'daki konteynerler aynı düğümdede çalışacak şekilde birlikte konumlandırılır ve birlikte planlanır.

6.1. Container Images (Konteyner İmajları)

Bir konteyner imajı, bir uygulamayı çalıştırmak için gereken her şeyi içeren, çalışmaya hazır bir yazılım paketidir: kod ve gerektirdiği çalışma zamanı, uygulama ve sistem kütüphaneleri ve gerekli ayarlar için varsayılan değerler.

Konteynerler durumsuz ve değiştirilemez olacak şekilde tasarlanmıştır: Zaten çalışan bir konteynerin kodunu değiştirmemelisiniz. Konteynerleştirilmiş bir uygulamanız varsa ve değişiklik yapmak istiyorsanız, doğru işlem, değişikliği içeren yeni bir imaj oluşturmak ve ardından güncellenmiş imajdan başlamak için konteyneri yeniden oluşturmaktır.

6.2. Container Runtime (Konteyner Çalışma Zamanı)

Kubernetes'in konteynerleri etkili bir şekilde çalıştırmasını sağlayan temel bir bileşendir. Kubernetes ortamında konteynerlerin yürütülmesini ve yaşam döngüsünü yönetmekten sorumludur.

Kubernetes, containerd, CRI-O ve Kubernetes CRI'nin (Konteyner Çalışma Zamanı Arayüzü) diğer tüm uygulamaları gibi konteyner çalışma zamanlarını destekler.

Genellikle, kümenizin bir Pod için varsayılan konteyner çalışma zamanını seçmesine izin verebilirsiniz. Kümenizde birden fazla konteyner çalışma zamanı kullanmanız gerekiyorsa, Kubernetes'in bu konteynerleri belirli bir konteyner çalışma zamanı kullanarak çalıştırmasını sağlamak için bir Pod için RuntimeClass belirtebilirsiniz. Ayrıca, farklı Pod'ları aynı konteyner çalışma zamanı ile ancak farklı ayarlarla çalıştırmak için RuntimeClass kullanabilirsiniz.

7 Workloads (İş Yükleri)

Kubernetes'te dağıtılabılır en küçük işlem nesnesi olan Pod'ları ve bunları çalıştırmanıza yardımcı olan daha üst düzey soyutlamaları anlayın.

Bir iş yükü, Kubernetes üzerinde çalışan bir uygulamadır. İş yükünüz tek bir bileşen veya birlikte çalışan birkaç bileşen olsun, Kubernetes'te onu bir dizi Pod içinde çalıştırırsınız. Kubernetes'te bir Pod, kümenizde çalışan bir veya daha fazla container kümesini temsil eder.

Kubernetes Pod'larının tanımlanmış bir yaşam döngüsü vardır. Örneğin, bir Pod kümenizde çalışmaya başladıktan sonra, o Pod'un çalıştığı düğümde kritik bir hata oluşması, o düğümdeki tüm Pod'ların başarısız olması anlamına gelir. Kubernetes bu seviyedeki hatayı son olarak ele alır: düğüm daha sonra sağlıklı hale gelse bile, kurtarmak için yeni bir Pod oluşturmanız gerekir.

Ancak, hayatı önemli ölçüde kolaylaştırmak için, her Pod'u doğrudan yönetmenize gerek yoktur. Bunun yerine, sizin adınıza bir dizi Pod'u yöneten iş yükü kaynaklarını kullanabilirsiniz. Bu kaynaklar, belirttiğiniz duruma uygun olarak doğru sayıda ve doğru türde Pod'un çalıştığından emin olan denetleyicileri yapılandırır.

Kubernetes, çeşitli yerleşik iş yükü kaynakları sağlar:

- **Deployment ve ReplicaSet:** Deployment, kümenizde durumsuz bir uygulama iş yükünü yönetmek için uygundur; burada Deployment'taki herhangi bir Pod gerekirse yenileriyle değiştirilebilir.

- **StatefulSet:** Bir şekilde durumu izleyen bir veya daha fazla ilgili Pod çalıştırmanıza olanak tanır. Örneğin, iş yükünüz verileri kalıcı olarak kaydediyorsa, her Pod'u bir PersistentVolume ile eşleştiren bir StatefulSet çalıştırabilirsiniz. Bu StatefulSet için Pod'larda çalışan kodunuz, genel dayanıklılığı artırmak için aynı StatefulSet'teki diğer Pod'lara veri kopyalayabilir.
- **DaemonSet:** düğümlere özgü tesisler sağlayan Pod'ları tanımlar. DaemonSet'te belirtilen özelliklere uyan bir düğümü kümenize her eklediğinizde, kontrol düzlemi bu DaemonSet için yeni düğümüne bir Pod planlar. Bir DaemonSet'teki her Pod, klasik bir Unix/POSIX sunucusundaki sistem daemon'una benzer bir iş gerçekleştirir. Bir DaemonSet, kümenizin çalışması için temel olabilir; örneğin, küme ağını çalıştırmak için bir eklenti olabilir, düğümü yönetmenize yardımcı olabilir veya çalıştırdığınız konteyner platformunu geliştiren isteğe bağlı davranışlar sağlayabilir.
- **Job ve CronJob:** Tamamlanana kadar çalışan ve ardından duran görevleri tanımlamanın farklı yollarını sunar. Bir Job kullanarak, yalnızca bir kez tamamlanana kadar çalışan bir görevi tanımlayabilirsiniz. Bir CronJob kullanarak, aynı Job'u bir programa göre birden fazla kez çalıştırabilirsiniz.

7.1. Pods

Podlar, Kubernetes'te oluşturabileceğiniz ve yönetebileceğiniz en küçük dağıtılabilir bilgi işlem birimleridir.

Bir Pod, paylaşılan depolama ve ağ kaynaklarına sahip bir veya daha fazla konteynerden oluşan bir gruptur ve konteynerlerin nasıl çalıştırılacağına dair bir spesifikasyona sahiptir. Bir Pod'un içeriği her zaman aynı yerde bulunur ve aynı zamanlamaya göre planlanır ve paylaşılan bir bağlamda çalışır. Bir Pod, uygulamaya özgü bir "mantıksal ana bilgisayar"ı modeller: nispeten sıkı bir şekilde birbirine bağlı bir veya daha fazla uygulama konteyneri içerir. Bulut dışı bağlamlarda, aynı fiziksel veya sanal makinede yürütülen uygulamalar, aynı mantıksal ana bilgisayarda yürütülen bulut uygulamalarına benzer.

Uygulama konteynerlerinin yanı sıra, bir Pod, Pod başlatma sırasında init container da içerebilir. Ayrıca, bir Pod'da hata ayıklama için ephemeral container da ekleyebilirsiniz.

7.1.1. Pod Nedir?

Pod'ların çalışabilmesi için kümedeki her düğüme bir container runtime kurmanız gerekir.

Bir Pod'un paylaşılan bağlamı, bir dizi Linux ad alanı, cgrup ve potansiyel olarak diğer izolasyon unsurlarından oluşur. Bir konteyneri izole eden şeylerle aynıdır. Bir Pod'un bağlamı içinde, bireysel uygulamalara daha fazla alt izolasyon uygulanabilir.

Bir Pod, paylaşılan ad alanlarına ve paylaşılan dosya sistemi birimlerine sahip bir konteyner kümesine benzer.

Kubernetes kümesindeki Pod'lar iki ana şekilde kullanılır:

- **Tek bir konteyner çalıştıran Pod'lar:** Pod başına bir konteyner" modeli, en yaygın Kubernetes kullanım durumudur; bu durumda, bir Pod'u tek bir konteynerin etrafındaki bir sarmalayıcı olarak düşünebilirsiniz; Kubernetes, konteynerleri doğrudan yönetmek yerine Pod'ları yönetir.
- **Birlikte çalışması gereken birden fazla konteyner çalıştıran pod'lar:** Bir Pod, birbirine sıkıca bağlı ve kaynakları paylaşması gereken, aynı yerde bulunan birden fazla konteynerden oluşan bir uygulamayı kapsayabilir. Bu aynı yerde bulunan konteynerler tek bir bütünleşik birim oluşturur.

Birden fazla aynı yerde bulunan ve birlikte yönetilen konteyneri tek bir Pod'da gruplandırmak nispeten gelişmiş bir kullanım durumudur. Bu modeli yalnızca konteynerlerinizin birbirine sıkıca bağlı olduğu belirli durumlarda kullanmalısınız.

7.1.2. Pod'ları Yönetmek İçin İş Yüğü Kaynakları

Genellikle, tekil Pod'lar da dahil olmak üzere Pod'ları doğrudan oluşturmanıza gerek yoktur. Bunun yerine, Deployment veya Job gibi iş yükü kaynaklarını kullanarak oluşturun. Pod'larınızın durumu izlemesi gerekiyorsa, StatefulSet kaynağını göz önünde bulundurun. Her Pod, belirli bir uygulamanın tek bir örneğini çalıştırmak için tasarlanmıştır. Uygulamanızı yatay olarak ölçeklendirmek (daha fazla örnek çalıştırarak daha fazla genel kaynak sağlamak) istiyorsanız, her örnek için bir tane olmak üzere birden fazla Pod kullanmalısınız. Kubernetes'te bu genellikle çoğaltma olarak adlandırılır.

Çoğaltılan Pod'lar genellikle bir iş yükü kaynağı ve denetleyicisi tarafından bir grup olarak oluşturulur ve yönetilir.